

1. Project Title

BITS Pilani – Work Integrated Learning Programmes

RecoMart Recommendation System

End-to-End Data Engineering and MLOps Pipeline

DM4DL Assignment

Report Version: 3.0

Academic Year: 2026

Generated: 2026-07-21T09:18:19Z

Generated by: RecoMart Project Report Generator

2. Team Member Details

S. No.	Name	BITS ID / Email ID
1	RAMAMURTHY S	2025ab05099@wilp.bits-pilani.ac.in
2	MEGHA ASHWIN KANU	2025ab05130@wilp.bits-pilani.ac.in
3	RANJITA PATEL	2025ab05117@wilp.bits-pilani.ac.in
4	KIRAN KUMAR DEVARA	2025ab05105@wilp.bits-pilani.ac.in
5	VIVEK RANJAN UPADHYAY	2025ab05127@wilp.bits-pilani.ac.in

Table of Contents

1. Project Title	1
2. Team Member Details	2
3. Problem Statement	3
4. Objectives	4
5. Methodology / Pipeline	5
6. Implementation Details	7
7. Results and Output Screenshots	16
8. Conclusion and Future Scope	32

3. Problem Statement

RecoMart is an enterprise-style data management and machine-learning pipeline for recommendation systems. It ingests MovieLens data, preserves immutable source records, validates and prepares datasets, builds reusable recommendation features, trains models, records experiments, and publishes auditable reports.

The repository is an academic project, but its design follows production principles: separation of concerns, reproducibility, traceability, secure configuration, automated testing, observable execution, and recoverable data processing.

RecoMart uses a layered batch architecture with ports and adapters. Pipeline stages communicate through immutable S3 objects and PostgreSQL metadata rather than hidden in-process state. Airflow supplies control flow; src/ supplies business behavior.

Every stage defines:

- accepted input layer and schema version;
- required identifiers and partition values;
- deterministic transformation behavior;
- output schema, format, and quality requirements;
- metadata and lineage records;
- error categories and retry behavior.

Contracts are versioned. Breaking schema changes require a new major schema version and an explicit migration or compatibility path.

The validation layer implements Assignment Part 4 between immutable raw storage and later preparation. It profiles and validates users, products, clickstream, purchasehistory, and popularity without changing the raw objects or imputing values. Business logic lives in src/validation; an Airflow DAG may invoke the runner but must not contain rules.

The modeling stage consumes immutable feature-store data and the chronological prepared splits referenced by the feature manifest. It never reads raw, quarantined, or unvalidated data. The command `python -m src.modeling.cli` executes one reproducible training run.

Apache Airflow coordinates the existing RecoMart application services. The DAG contains no generation, ingestion, validation, transformation, feature, lineage, or modeling business logic. Each task calls the corresponding runner and treats its manifest as the durable stage contract.

The primary DAG ID is `recomartendtoendpipeline`. It is manual by default, uses UTC, has `catchup=False`, and limits concurrency to one active DAG run. `RECOMARTPIPELINESCHEDULE` may supply a cron expression without changing code.

4. Objectives

1. - Provide a FastAPI service that creates controlled synthetic data batches. - Store every batch in an S3-compatible data lake. - Use Apache Airflow solely to schedule and orchestrate processing. - Keep business logic in independently testable modules under src/. - Validate data contracts before allowing data into downstream layers. - Produce deterministic prepared datasets and features. - Maintain a feature store with point-in-time and version metadata. - Capture end-to-end lineage for data, features, models, and reports. - Train and evaluate recommendation models reproducibly. - Track parameters, metrics, artifacts, and model versions in MLflow. - Generate human-readable data-quality and model-performance reports.

5. Methodology / Pipeline

5.1 Pipeline Flow

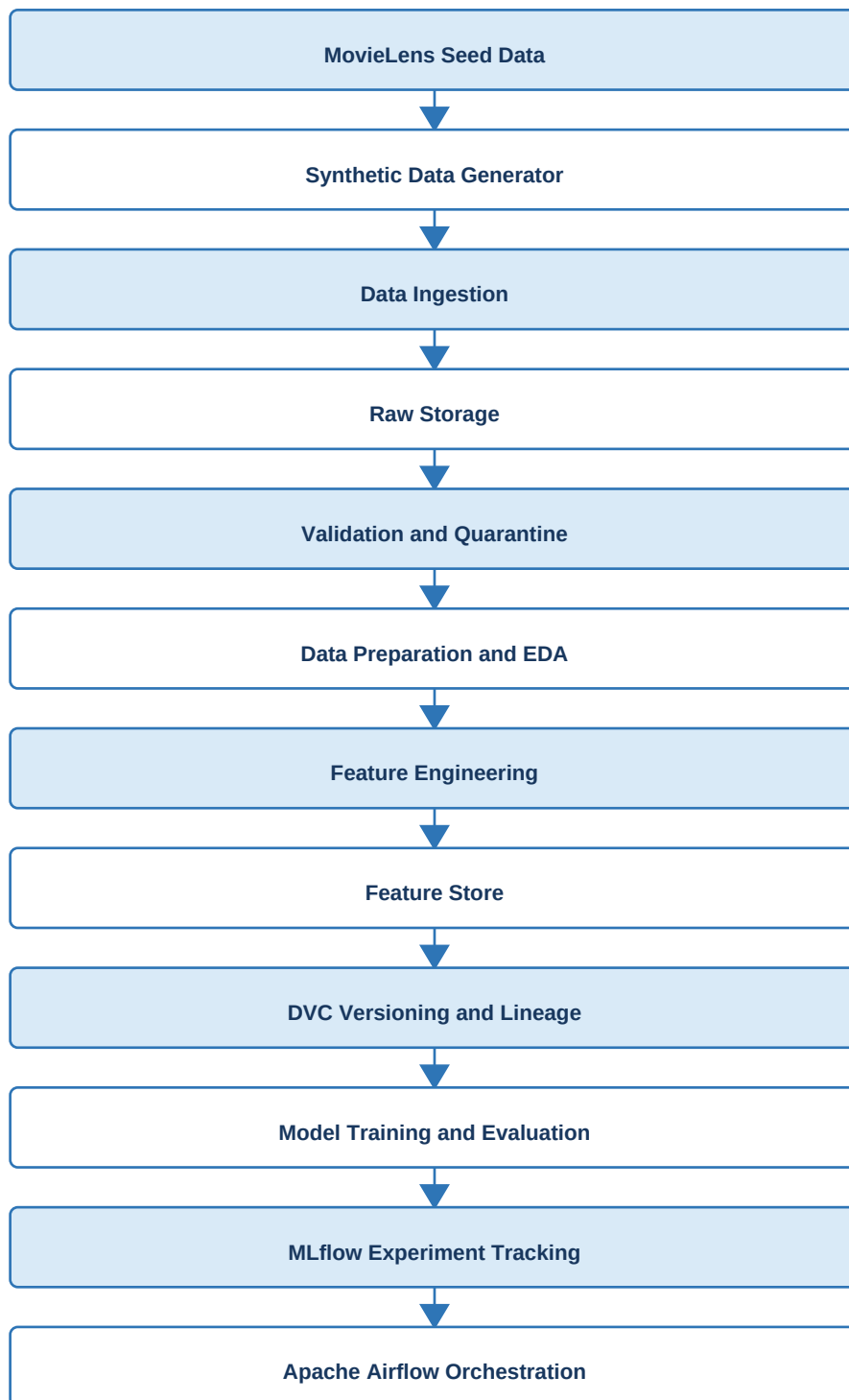


Figure 1. RecoMart pipeline flow derived from repository architecture.

5.2 Data Sources

RecoMart ingestion moves generated source data from the incoming layer to an immutable raw layer. It implements Assignment Part 2 (data collection and ingestion) and Part 3 (raw data storage). One invocation performs one bounded batch and exits; scheduling belongs to Airflow.

The ingestion path is:

No validation, cleaning, schema conversion, feature engineering, or model logic is performed here. CSV remains CSV, JSON remains JSON, and source bytes are preserved.

5.3 Storage Zones

5.4 Execution Flow

The DAG uses direct Python application services because the existing runners have stable typed APIs and return manifest data. `src/orchestration/stages.py` adapts each return value to a compact JSON-safe contract. It does not reproduce stage calculations. This preserves in-process exception context and avoids parsing console output.

Feature calculation and feature-store persistence remain one task because the existing `FeatureRunner` commits both transactionally. Versioning has two orchestration steps: data artifacts are registered before training, then the model and model-report versions are registered after training so the final lineage graph can truthfully include artifacts that now exist.

6. Implementation Details

6.1 Synthetic Data Generation

- Movie, rating, user, and related MovieLens records. - Synthetic batch creation without changing the source meaning. - Batch ingestion, schema and business-rule validation. - Data preparation and recommendation-oriented feature engineering. - Offline model training and evaluation. - Dataset, feature, model, and report lineage. - Operational metadata and run status.

6.2 Data Ingestion

`src.ingestion.fileingestion.FileIngestionService` handles the four required files:

- `users.csv` - `products.json` - `clickstream.csv` - `purchasehistory.csv`

For each file it verifies existence and readability, calculates SHA-256, counts records, builds the canonical raw partition, publishes through the storage interface, and returns manifest metadata. CSV counting excludes the header. JSON input must be a top-level array. A missing required source is an explicit `SourceFileNotFoundError` and is never silently skipped or retried.

`src.ingestion.apiingestion.PopularityApiIngestionService` retrieves product popularity strictly through HTTP. It does not read `popularity.json` directly. The default source is:

The HTTP response must be a JSON array whose records contain:

- `productid` - `averagerating` - `totalratings` - `popularityscore` - `trend` - `updatedat`

The complete successful response bytes are checksummed and written as `popularity.json` under the API raw partition.

`src.api.main` exposes `GET /api/v1/popularity`. The route is thin and delegates to `PopularityService`, which joins generated `products.json` aggregate rating fields with generated `popularity.json` scores and trends. Supported query parameters are:

6.3 Storage Structure

6.4 Data Validation and Quarantine

Common checks cover file shape, required and unexpected columns, required values, entire duplicates, and business-key uniqueness. Dataset validators add configured ID, range, UUID, enum, date, rating, and business-contract rules. Relationship rules use the loaded raw parent datasets, so a quality problem in a parent record does not falsely become a missing-reference problem in every child. Orphans in clickstream, purchases, or popularity are ERROR failures.

Purchase amounts are compared to `quantity products.price`. Purchase chronology uses the strongest available link: the latest clickstream timestamp for the same user-product pair. MovieLens-derived files do not retain an exact event-to-order identifier, so absence of such a pair is a warning rather than a fabricated correlation. Explicit ratings exist only in purchase history; products and popularity retain decimal aggregate ratings.

A row enters data/validated only when it passes every record-level ERROR rule. A row failing one or more ERROR rules enters data/quarantine unchanged except for appended trace fields: error codes, error messages, validation run ID, batch ID, and quarantine timestamp. WARNING rules remain reportable but do not quarantine otherwise valid records. Each dataset also receives validationerrors.json, including dataset-level and skipped-rule diagnostics.

6.5 Data Preparation and EDA

Column names and whitespace are standardized, configured categorical casing is normalized, numerical types are enforced, and dates/timestamps are parsed as UTC. IDs are preserved. Events and purchases are sorted chronologically. Escaped exact duplicates are removed and counted; no removal is silent. Required values are not imputed. Optional product release dates remain null. Unavailable validated popularity enrichment leaves popularityscore null and uses the configured Unknown trend, with a manifest warning.

Gender, occupation, customer segment, category, and trend use one-hot encoding with an explicit future-safe Unknown category. Brand uses frequency encoding because its meaning is nominal and its cardinality may grow. Original fields are retained. Metadata JSON records categories, frequencies, and unknown behavior.

StandardScaler is used for continuous fields. Skewed nonnegative counts and spend/implicit aggregates use log1p then StandardScaler. Missing optional numeric values remain null and are excluded from fitting. Metadata records method, fitted count, mean, and scale.

UTC interaction timestamps derive hour, day-of-week, day, month, weekend, recency, and sine/cosine cyclical features. User registration and product age are derived relative to the UTC run start. Interactions are globally ordered by timestamp and stable interaction ID. Earliest 70% is training, next 15% validation, and latest 15% test; integer rounding assigns the remainder to training. Tiny valid fixtures reserve at least one validation and test row. Split boundaries and cold-start entity counts are recorded.

6.6 Feature Engineering

User features cover lifetime/windowed activity, product/session diversity, ratings, commerce, recency, tenure, preferences, conversions, quantile activity level, and cold start. Item features cover activity, unique users, explicit ratings, commerce, separate popularity measures, windows/growth, catalog attributes, ranks, long tail, and cold start. User-item features contain observed-pair counts, implicit/explicit feedback, temporal history, affinities, and price preference.

Sparse binary user-item multiplication creates canonical co-occurrence pairs without a Cartesian product. Support, directional confidence, and lift use distinct active users. Same-session and co-purchase counts are separate.

Sparse weighted item-user vectors feed brute-force cosine nearest neighbors. Jaccard uses user-set overlap. Content similarity covers category, brand, normalized price distance, and rating distance. Configured weights combine the six measures and retain top-K non-self neighbors.

Features | Type | Logic / source | Null/default | ---|---|---|---| totalinteractions, uniqueproductsinteracted | integer | Count events/distinct products | 0 | viewcount, wishlistcount,

addtocartcount, purchasecount | integer | Event-type counts | 0 | interactionfrequencyperday | float | total / max(1, observation days) | 0 | interactionfrequencylast7d, interactionfrequencylast30d | float | Window count / window days | 0 | activedays, sessioncount, averageeventspersession | numeric | Distinct UTC dates/sessions and safe ratio | Ratio null if no sessions | averageratinggiven, ratingcount, ratingstddev, minimumratinggiven, maximumratinggiven | numeric | Valid non-null explicit ratings only | Statistics null without ratings; count 0 | totalquantitypurchased, totalspend | float | Purchase sums | 0 | averageordervalue, maximumordervalue | float | Purchase amounts | Null without purchase | dayssincelastinteraction, dayssincelastpurchase | float | Reference minus latest UTC event | Purchase recency null without purchase | usertenuredays | float | Reference minus registration | Null if registration unavailable | preferredcategory, preferredbrand | string | Highest implicit-weight total | Null without events | preferredinteractiontype | string | Highest event count | Null without events | viewtocartratio, carttopurchaseratio, purchaseconversionrate | float | Safe directional ratios | Null on zero denominator | useractivitylevel | string | LOW < q33; HIGH > q66; else MEDIUM | Thresholds persisted | coldstartuserflag | boolean | total interactions < configured 5 | False/true |

Activity/type/unique-user counts, rating statistics, quantity/revenue/order amount, three separate popularity scores, conversion ratios, 7/30-day counts, growth rate, category/brand/price/scaled price/product age, overall/category rank, long-tail flag, and cold-start flag are materialized. Ratings remain null without explicit feedback. Long tail means interaction count below the 80th percentile boundary. Cold start means unique users below 3 or interactions below 5.

Observed-pair interaction/type counts, implicit score, explicit/latest/average rating, quantity/spend, first/last interaction and purchase timestamps, reference-time recencies, average interval, sessions, safe conversion ratios, category affinity, brand affinity, price preference distance, and $1/(1+distance)$ similarity are materialized. Affinity is category/brand implicit weight divided by the user's total implicit weight.

Canonical itemida < itemidb; distinct shared-user count, shared-session count, co-purchase count, item user counts, support, both confidences, and lift. Pairs require configured count ≥ 2 and support ≥ 0.001 , with at most 100 retained neighbors per endpoint.

Cosine implicit-vector similarity, Jaccard user-set similarity, binary category/brand matches, price similarity, rating similarity, weighted combined score, and per-product rank. Combined weights are 0.45/0.20/0.15/0.05/0.10/0.05. Self-pairs and scores below 0.01 are excluded; top 50 per product remain.

6.7 Feature Store

Table	Row Count	Columns	Primary Key
feature_batch	3	feature_batch_id (VARCHAR(80)), source_batch_id (VARCHAR(100)), preparation_run_id (VARCHAR(80)), feature_reference_timestamp (TIMESTAMP), feature_source_split (VARCHAR(10)), started_at (TIMESTAMP), completed_at (TIMESTAMP), status (VARCHAR(30)), user_feature_count (BIGINT), item_feature_count (BIGINT), user_item_feature_count (BIGINT), cooccurrence_feature_count (BIGINT), similarity_feature_count (BIGINT), configuration_hash (VARCHAR(64)), source_checksum (VARCHAR(64)), identity_hash (VARCHAR(64)), error_message (TEXT), created_at (TIMESTAMP)	feature_batch_id
feature_definition	94	feature_name (VARCHAR(150)), feature_group (VARCHAR(50)), entity_type (VARCHAR(50)), data_type (VARCHAR(50)), description (TEXT), calculation_logic (TEXT), source_columns (TEXT), default_value (TEXT), null_handling (TEXT), feature_version (VARCHAR(30)), owner (VARCHAR(80)), is_active (BOOLEAN), created_at (TIMESTAMP), updated_at (TIMESTAMP)	feature_name, feature_version
feature_lineage	282	lineage_id (VARCHAR(80)), feature_batch_id (VARCHAR(80)), feature_name (VARCHAR(150)), source_dataset (VARCHAR(150)), source_columns (TEXT), transformation_name (VARCHAR(150)), transformation_version (VARCHAR(30)), transformation_parameters (TEXT), source_checksum (VARCHAR(64)), output_table (VARCHAR(100)), generated_at (TIMESTAMP)	lineage_id
item_cooccurrence_features	469638	feature_batch_id (TEXT), source_batch_id (TEXT), feature_reference_timestamp (TEXT), item_id_a (BIGINT), item_id_b (BIGINT), cooccurrence_count (BIGINT), cooccurring_user_count (BIGINT), item_a_user_count (BIGINT), item_b_user_count (BIGINT), cooccurring_session_count (BIGINT), co_purchase_count (BIGINT), support (FLOAT), confidence_a_to_b (FLOAT), confidence_b_to_a (FLOAT), lift (FLOAT), created_at (TEXT)	None
item_features	5046	feature_batch_id (TEXT), source_batch_id (TEXT), feature_reference_timestamp (TEXT), product_id (BIGINT), category (TEXT), brand (TEXT), price (FLOAT), price_scaled (FLOAT), product_age_days (FLOAT), total_interactions (FLOAT), unique_users (FLOAT), average_rating (FLOAT), rating_count (FLOAT), rating_stddev (FLOAT), minimum_rating (FLOAT), maximum_rating (FLOAT), total_quantity_sold (FLOAT), total_revenue (FLOAT), average_selling_amount (FLOAT), view_count (FLOAT), wishlist_count (FLOAT), add_to_cart_count (FLOAT), purchase_count (FLOAT), interaction_popularity_score (FLOAT), purchase_popularity_score (FLOAT), external_popularity_score (FLOAT), view_to_cart_ratio (FLOAT), cart_to_purchase_ratio (FLOAT), purchase_conversion_rate (FLOAT), interaction_count_last_7d (FLOAT), interaction_count_last_30d (FLOAT), popularity_growth_rate (FLOAT), item_popularity_rank (BIGINT), category_popularity_rank (BIGINT), long_tail_flag (BOOLEAN), cold_start_item_flag (BOOLEAN), created_at (TEXT)	None
item_similarity_features	231250	feature_batch_id (TEXT), source_batch_id (TEXT), feature_reference_timestamp (TEXT), product_id (BIGINT), similar_product_id (BIGINT), cosine_similarity (FLOAT), jaccard_similarity (FLOAT), category_similarity (FLOAT), brand_similarity (FLOAT), price_similarity (FLOAT), rating_similarity (FLOAT), combined_similarity_score (FLOAT), similarity_rank (BIGINT), created_at (TEXT)	None

Table	Row Count	Columns	Primary Key
user_features	2828	feature_batch_id (TEXT), source_batch_id (TEXT), feature_reference_timestamp (TEXT), user_id (BIGINT), total_interactions (BIGINT), unique_products_interacted (BIGINT), active_days (BIGINT), session_count (BIGINT), average_rating_given (FLOAT), rating_count (BIGINT), rating_stddev (FLOAT), minimum_rating_given (FLOAT), maximum_rating_given (FLOAT), total_quantity_purchased (FLOAT), total_spend (FLOAT), average_order_value (FLOAT), maximum_order_value (FLOAT), view_count (BIGINT), wishlist_count (BIGINT), add_to_cart_count (BIGINT), purchase_count (BIGINT), interaction_frequency_per_day (FLOAT), interaction_frequency_last_7d (FLOAT), interaction_frequency_last_30d (FLOAT), average_events_per_session (FLOAT), days_since_last_interaction (FLOAT), days_since_last_purchase (FLOAT), user_tenure_days (FLOAT), preferred_category (TEXT), preferred_brand (TEXT), preferred_interaction_type (TEXT), view_to_cart_ratio (FLOAT), cart_to_purchase_ratio (FLOAT), purchase_conversion_rate (FLOAT), user_activity_level (TEXT), cold_start_user_flag (BOOLEAN), created_at (TEXT)	None
user_item_features	134785	feature_batch_id (TEXT), source_batch_id (TEXT), feature_reference_timestamp (TEXT), user_id (BIGINT), product_id (BIGINT), interaction_count (BIGINT), implicit_score (FLOAT), average_explicit_rating (FLOAT), latest_explicit_rating (FLOAT), total_quantity (FLOAT), total_spend (FLOAT), first_interaction_timestamp (TIMESTAMP), last_interaction_timestamp (TIMESTAMP), session_count (BIGINT), view_count (BIGINT), wishlist_count (BIGINT), add_to_cart_count (BIGINT), purchase_count (BIGINT), first_purchase_timestamp (TIMESTAMP), last_purchase_timestamp (TIMESTAMP), explicit_rating (FLOAT), days_since_last_interaction (FLOAT), days_since_last_purchase (FLOAT), average_time_between_interactions_hours (FLOAT), user_item_view_to_cart_ratio (FLOAT), user_item_cart_to_purchase_ratio (FLOAT), user_category_affinity (FLOAT), user_brand_affinity (FLOAT), price_preference_distance (FLOAT), price_preference_similarity (FLOAT), created_at (TEXT)	None

Sample rows: item_cooccurrence_features

feature_batch_id	source_batch_id	feature_reference_timestamp	item_id_a	item_id_b	cooccurrence_count
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	151	37
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	50	31
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	121	31
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	100	28
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	405	28

Sample rows: item_similarity_features

feature_batch_id	source_batch_id	feature_reference_timestamp	product_id	similar_product_id	cosine_similarity
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	588	0.19196625094719144
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	501	0.15282336190273427
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	946	0.1761400250531574
FEAT_20260718_22_4621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	969	0.15713739036217866

feature_batch_id	source_batch_id	feature_reference_timestamp	product_id	similar_product_id	cosine_similarity
FEAT_20260718_224621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	240	0.1766656633494249

Sample rows: user_item_features

feature_batch_id	source_batch_id	feature_reference_timestamp	user_id	product_id	interaction_count
FEAT_20260718_224621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	11	1
FEAT_20260718_224621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	13	4
FEAT_20260718_224621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	15	4
FEAT_20260718_224621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	21	1
FEAT_20260718_224621_d4f3c8	RECO_20260718_211004_2ec353	2026-07-18T19:12:24+00:00	1	23	3

6.8 Data Versioning and Lineage

RecoMart versions data because source code alone cannot reproduce a model. Reproducibility requires the exact input bytes, transformation configuration, manifests, feature materialization, model artifacts, and reports. Git versions source code, YAML, SQL, DVC metadata, and documentation. DVC versions generated and large artifacts. Git LFS is not used.

Child | Transformation | ---|---| Raw | Immutable file and REST API ingestion | Validated | Schema, quality, range, format, and referential validation | Prepared | Cleaning, encoding, normalization, interactions, matrices, split | Features | Point-in-time recommendation feature engineering | Models | Collaborative and content-based training | EDA reports | Exploratory data analysis | Model reports | Evaluation and comparison reporting | MLflow runs | Parameters, metrics, models, and artifacts logged per algorithm |

Transformation configuration remains Git-versioned and its SHA-256 is copied from the stage manifest where available.

Stage	Command	Dependencies	Outputs
generator	python -m src.versioning.dvc_stage --stage incoming --output reports/versioning/stages/incoming.json	src/generator src/versioning data/incoming configs/versioning.yaml	reports/versioning/stages/incoming.json
ingestion	python -m src.versioning.dvc_stage --stage raw --output reports/versioning/stages/raw.json	reports/versioning/stages/incoming.json src/ingestion data/raw configs/versioning.yaml	reports/versioning/stages/raw.json
validation	python -m src.versioning.dvc_stage --stage validated --output reports/versioning/stages/validated.json	reports/versioning/stages/raw.json src/validation data/validated reports/data_quality configs/versioning.yaml	reports/versioning/stages/validated.json
preparation	python -m src.versioning.dvc_stage --stage prepared --output reports/versioning/stages/prepared.json	reports/versioning/stages/validated.json src/preparation data/prepared configs/versioning.yaml	reports/versioning/stages/prepared.json

Stage	Command	Dependencies	Outputs
feature_engineering	<code>python -m src.versioning.dvc_stage --stage features --output reports/versioning/stages/features.json</code>	reports/versioning/stages/prepared.json src/feature_engineering_data/features configs/versioning.yaml	reports/versioning/stages/features.json
training	<code>python -m src.versioning.dvc_stage --stage models --output reports/versioning/stages/models.json</code>	reports/versioning/stages/features.json src/modeling_models reports/model_training configs/versioning.yaml	reports/versioning/stages/models.json
registry_and_lineage	<code>python -m src.versioning.cli</code>	src/versioning reports/versioning/stages/incoming.json reports/versioning/stages/raw.json reports/versioning/stages/validated.json reports/versioning/stages/prepared.json reports/versioning/stages/features.json reports/versioning/stages/models.json reports/eda configs/versioning.yaml	{'reports/versioning/dataset_registry.json': {'persist': True}} reports/versioning/lineage_report.json reports/versioning/version_summary.json reports/versioning/pipeline_lineage.png

6.9 Model Training and Evaluation

The implementation is deterministic biased Funk SVD, a matrix-factorization form of collaborative filtering:

$$\text{rating}(u,i) = \text{globalmean} + \text{userbias} + \text{itembias} + \text{userfactors} \cdot \text{itemfactors}$$

Stochastic gradient descent optimizes squared error with L2 regularization. Only valid, observed explicit ratings are used; absent interactions are not converted to negative ratings. Unknown users or items fall back to known biases and the global mean.

Product category and brand are one-hot encoded. Price and average rating are standardized. Rows are L2-normalized and compared with cosine similarity. User recommendations use the centroid of products in the user's training history and exclude already-observed products. `similaritems` compares one catalog item to all other items and excludes self-pairs.

Feature importance is reported as the proportion of transformed dimensions assigned to category, brand, price, and rating. This describes representation capacity rather than causal importance.

Metric | Meaning | ---|---| RMSE | Square-root mean squared explicit-rating error; collaborative only. | MAE | Mean absolute explicit-rating error; collaborative only. | Precision@K | Relevant recommended items divided by K. | Recall@K | Relevant recommended items divided by relevant test items. | MAP@K | Mean average precision, rewarding early relevant hits. | NDCG@K | Position-discounted ranking quality normalized per user. | Hit Rate@K | Fraction of users with at least one relevant recommendation. | Coverage | Unique recommended products divided by catalog size. | Catalog Coverage | Alias reported explicitly for content evaluation. | Intra-list Diversity | One minus mean category/brand equality similarity. | Novelty | Mean $-\log_2(\text{item interaction probability})$ of recommendations. | Similarity Quality | Mean cosine score of example similar-item results. |

Undefined metrics are represented as null/NaN and are not coerced to zero. RMSE and MAE are not applicable to the content model because it does not predict an explicit rating.

Model	Run ID	Available Metrics	Status
collaborative	32228e20d83348549b16e5141f1f9135	rmse=0.4585, mae=0.4089, precision_at_10=0.0452, recall_at_10=0.0383, map_at_10=0.0183, ndcg_at_10=0.0490, hit_rate_at_10=0.3236, coverage=0.0904, diversity=0.8850, novelty=3.2206	SUCCESS
content_based	9a6d722cb7574f7093830410a4e7842b	precision_at_10=0.0164, recall_at_10=0.0133, map_at_10=0.0055, ndcg_at_10=0.0168, hit_rate_at_10=0.1401, coverage=0.5208, diversity=0.9699, novelty=5.6655	SUCCESS

6.10 MLflow Experiment Tracking

The modeling pipeline uses the URI in MLFLOWTRACKINGURI when set. Otherwise it uses the configured local mlruns/ store. MLflow 3.x local-file tracking is explicitly opted in for this assignment fallback. A remote HTTP tracking server can be used without code changes.

Parameters include algorithm settings, feature batch, source batch, dataset sizes, K, and configuration hash. Metrics include accuracy, ranking, coverage, diversity, novelty, duration, and memory-related summaries where applicable. Artifacts include serialized models, evaluation JSON, metadata, and the effective training configuration.

MLflow returns runid, experimentid, and artifacturi; these are copied into the training summary and model metadata.

Experiment / Run	Run ID	Parameters	Metrics	Status / Start
recomart-recommendation-models / MODEL_20260718_231246_7a56d5-content	448b5f7fa1b94b578ceb8172f4f7aab3	algorithm=cosine_content, catalog_size=1682, feature_s=category,brand,price,average_rating, feature_batch_id=FEAT_20260718_224621_d4f3c8, top_k=20	catalog_coverage=0.6373, coverage=0.6373, diversity=0.9821, evaluated_users=751, hit_rate_at_10=0.03595, inference_time_seconds=0.8641, intra_list_diversity=0.9821, map_at_10=0.002398	FINISHED 2026-07-18T23:12:53.469000+00:00
recomart-recommendation-models / MODEL_20260718_231246_7a56d5-collaborative	4153587f54d04076b67dd0d3867f9c41	algorithm=FunkSVD, epochs=20, factors=40, feature_batch_id=FEAT_20260718_224621_d4f3c8, learning_rate=0.005, random_seed=42, regularization=0.02, train_rows=61778	catalog_coverage=0.05767, coverage=0.05767, diversity=0.9598, evaluated_users=751, hit_rate_at_10=0.1332, inference_time_seconds=2.997, intra_list_diversity=0.9598, mae=0.4169	FINISHED 2026-07-18T23:12:52.412000+00:00

6.11 Apache Airflow Orchestration

Apache Airflow coordinates the existing RecoMart application services. The DAG contains no generation, ingestion, validation, transformation, feature, lineage, or modeling business logic. Each task calls the corresponding runner and treats its manifest as the durable stage contract.

The primary DAG ID is recomartendtoendpipeline. It is manual by default, uses UTC, has catchup=False, and limits concurrency to one active DAG run. RECOMARTPIPELINESCHEDULE may supply a cron expression without changing code.

Stage | Retries | Delay | Timeout | ---|---:|---:|---:| Generator | 1 | 1 minute | 10 minutes | Ingestion | 2 | 2 minutes | 15 minutes | Validation | 1 | default | 20 minutes | Preparation | 1 | default | 30 minutes

| Feature engineering/store | 1 | default | 45 minutes | Versioning | 1 | default | 15 minutes | Modeling | 1 | 5 minutes | 60 minutes | Summary | 0 | none | 5 minutes |

Application retries handle recoverable HTTP/S3 operations. Airflow retries recover whole tasks after infrastructure failure; deterministic quality-gate failures are not retried.

For production, create optional pools iopool (2 slots), databasepool (1 slot), and modeltrainingpool (1 slot), then assign those pool names through deployment policy. The local DAG does not require pre-created pools.

DAG	File	Task Sequence	Retry Policy
recomart_end_to_end_pipeline	dags/recomart_end_to_end_dag.py	configuration $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ generation $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ ingestion $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ validation $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ quality gate $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ preparation $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ feature store $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ versioning $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ modeling $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ lineage $\tilde{f}\tilde{A}\tilde{C}\tilde{A},\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$ summary	Stage-specific bounded retries (0 $\tilde{f}\tilde{A}\tilde{C}\tilde{A}\tilde{C}\tilde{A}\neg\tilde{A}\tilde{C}\tilde{A},\neg\tilde{a},\tilde{C}$, $\neg\tilde{A}^2$) and execution timeouts

7. Results and Output Screenshots

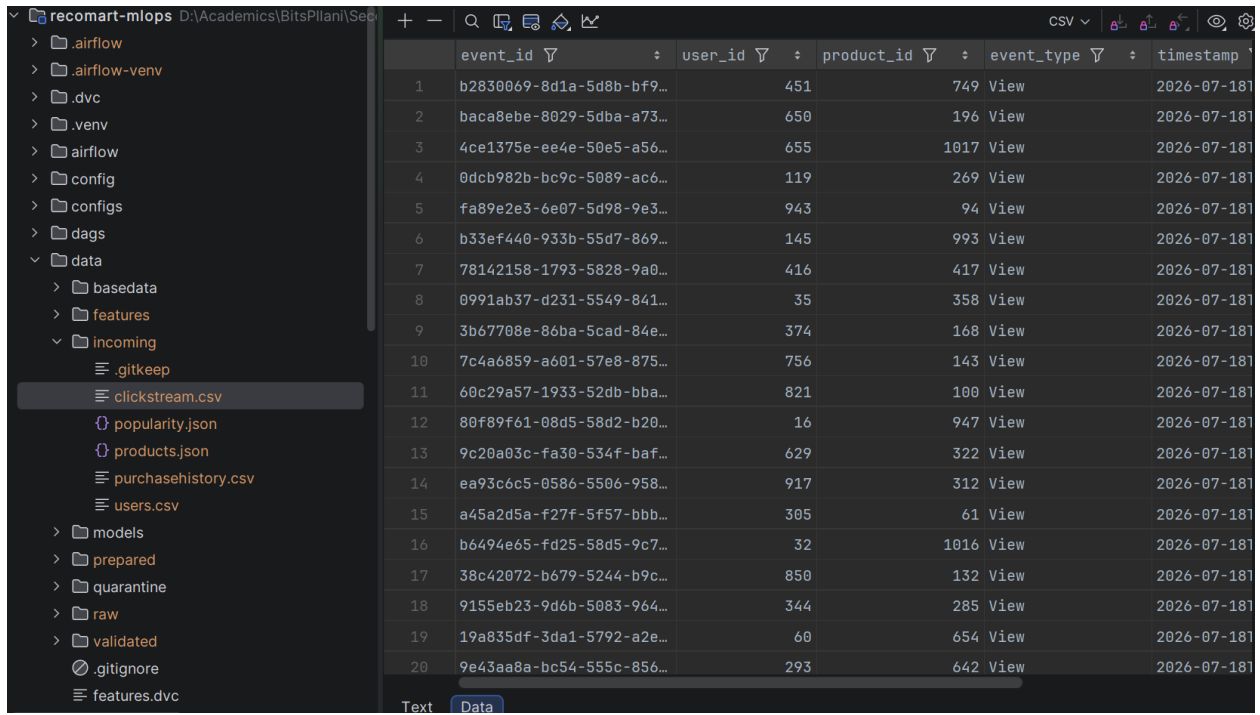
7.1 Dataset Summary

Dataset	Rows	Columns	Location
users	943	7	data/incoming/users.csv
products	1682	8	data/incoming/products.json
clickstream	176576	6	data/incoming/clickstream.csv
purchasehistory	55375	7	data/incoming/purchasehistory.csv
popularity	1682	4	data/incoming/popularity.json

7.2 Pipeline Evidence

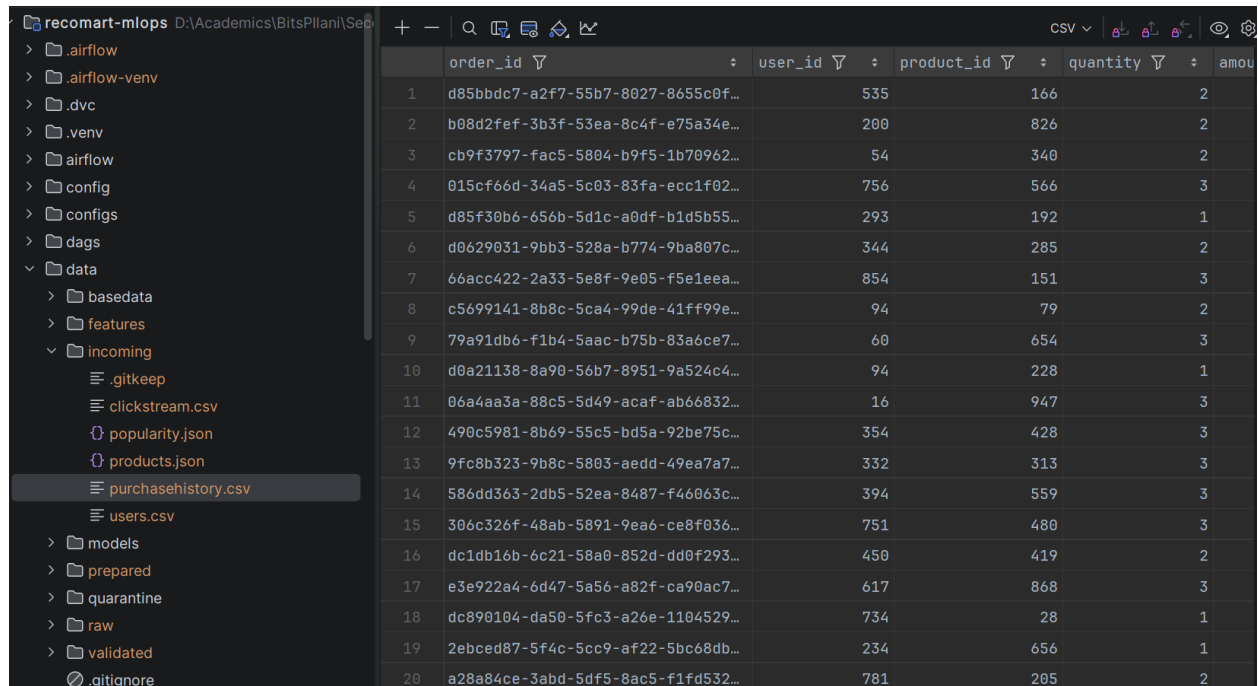
Stage	Evidence	Status
EDA	RECO_20260720_114419_07a4ea	Available
Feature Store	data/recomart_features.db	Available
DVC	7 stages in dvc.yaml	Available
Modeling	MODEL_20260720_114606_6e383a	SUCCESS
MLflow	2 selected successful runs	Available
Screenshots	6 matched images	Available

7.3 Repository Screenshots



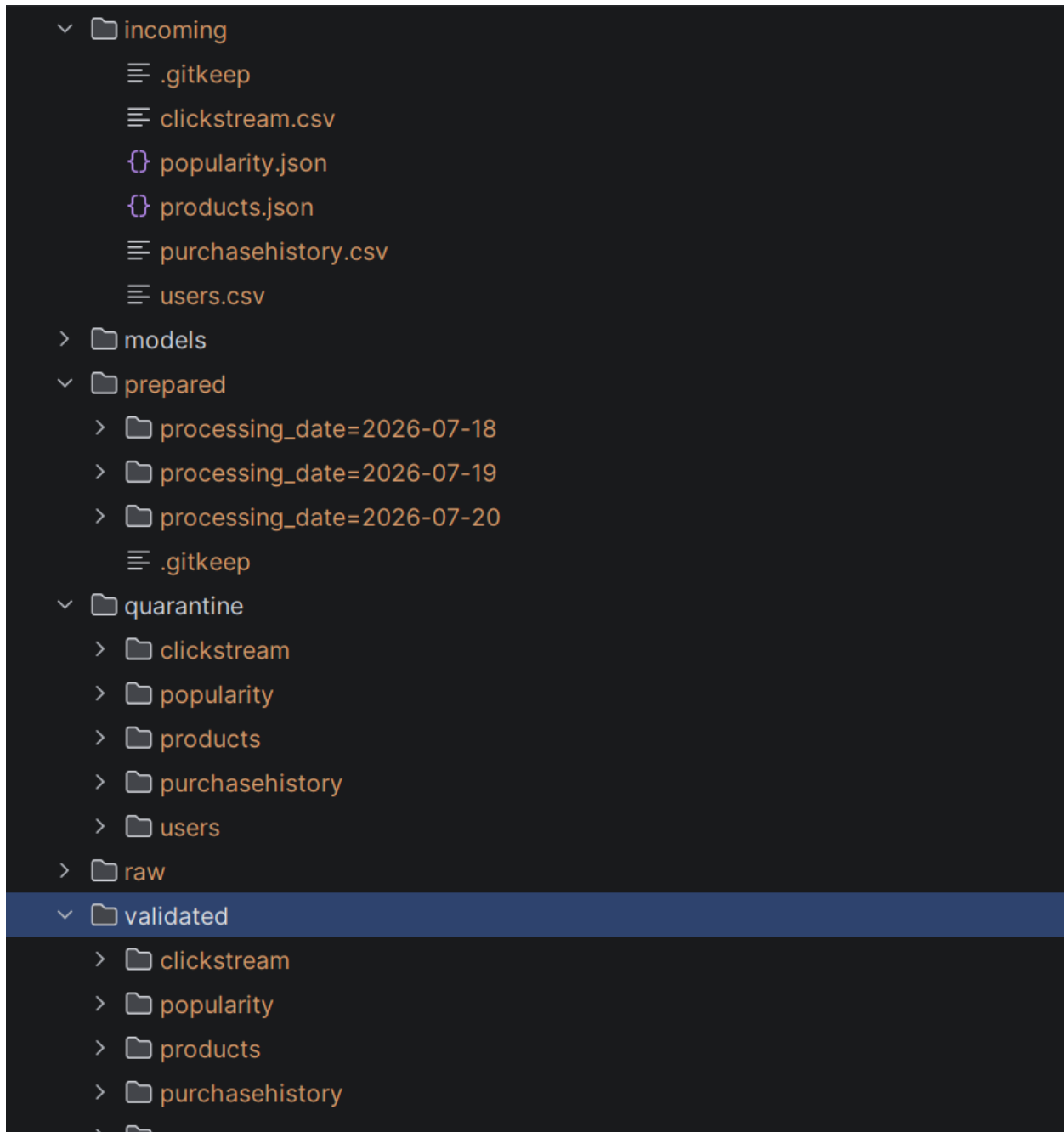
	event_id	user_id	product_id	event_type	timestamp
1	b2830069-8d1a-5d8b-bf9...	451	749	View	2026-07-181
2	baca8ebe-8029-5dba-a73...	650	196	View	2026-07-181
3	4ce1375e-ee4e-50e5-a56...	655	1017	View	2026-07-181
4	0dc982b-bc9c-5089-ac6...	119	269	View	2026-07-181
5	fa89e2e3-6e07-5d98-9e3...	943	94	View	2026-07-181
6	b33ef440-933b-55d7-869...	145	993	View	2026-07-181
7	78142158-1793-5828-9a0...	416	417	View	2026-07-181
8	0991ab37-d231-5549-841...	35	358	View	2026-07-181
9	3b67708e-86ba-5cad-84e...	374	168	View	2026-07-181
10	7c4a6859-a601-57e8-875...	756	143	View	2026-07-181
11	60c29a57-1933-52db-bba...	821	100	View	2026-07-181
12	80f89f61-08d5-58d2-b20...	16	947	View	2026-07-181
13	9c20a03c-fa30-534f-baf...	629	322	View	2026-07-181
14	ea93c6c5-0586-5506-958...	917	312	View	2026-07-181
15	a45a2d5a-f27f-5f57-bbb...	305	61	View	2026-07-181
16	b6494e65-fd25-58d5-9c7...	32	1016	View	2026-07-181
17	38c42072-b679-5244-b9c...	850	132	View	2026-07-181
18	9155eb23-9d6b-5083-964...	344	285	View	2026-07-181
19	19a835df-3da1-5792-a2e...	60	654	View	2026-07-181
20	9e43aa8a-bc54-555c-856...	293	642	View	2026-07-181

Clickstream Dataset 02_datasets_clickstream.png



	order_id	user_id	product_id	quantity	amou
1	d85bbdc7-a2f7-55b7-8027-8655c0f...	535	166	2	
2	b08d2fef-3b3f-53ea-8c4f-e75a34e...	200	826	2	
3	cb9f3797-fac5-5804-b9f5-1b70962...	54	340	2	
4	015cf66d-34a5-5c03-83fa-ecc1f02...	756	566	3	
5	d85f30b6-656b-5d1c-a0df-b1d5b55...	293	192	1	
6	d0629031-9bb3-528a-b774-9ba807c...	344	285	2	
7	66acc422-2a33-5e8f-9e05-f5e1eea...	854	151	3	
8	c5699141-8b8c-5ca4-99de-41ff99e...	94	79	2	
9	79a91db6-f1b4-5aac-b75b-83a6ce7...	60	654	3	
10	d0a21138-8a90-56b7-8951-9a524c4...	94	228	1	
11	06a4aa3a-88c5-5d49-acaf-ab66832...	16	947	3	
12	490c5981-8b69-55c5-bd5a-92be75c...	354	428	3	
13	9fc8b323-9b8c-5803-aedd-49ea7a7...	332	313	3	
14	586dd363-2db5-52ea-8487-f46063c...	394	559	3	
15	306c326f-48ab-5891-9ea6-ce8f036...	751	480	3	
16	dc1db16b-6c21-58a0-852d-dd0f293...	450	419	2	
17	e3e922a4-6d47-5a56-a82f-ca90ac7...	617	868	3	
18	dc890104-da50-5fc3-a26e-1104529...	734	28	1	
19	2ebced87-5f4c-5cc9-af22-5bc68db...	234	656	1	
20	a28a84ce-3abd-5df5-8ac5-f1fd532...	781	205	2	

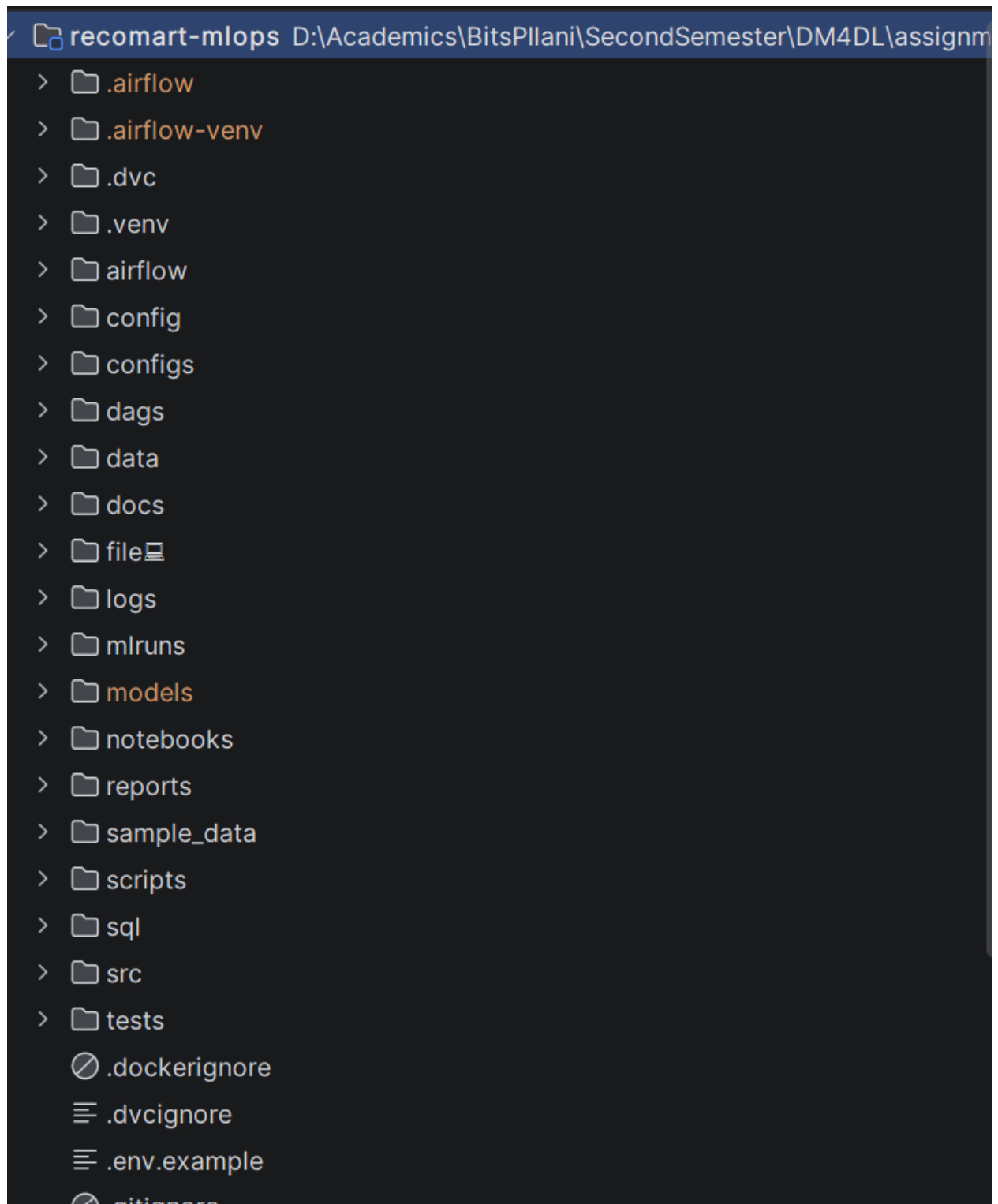
Purchase History Dataset 02_datasets_purchase.png



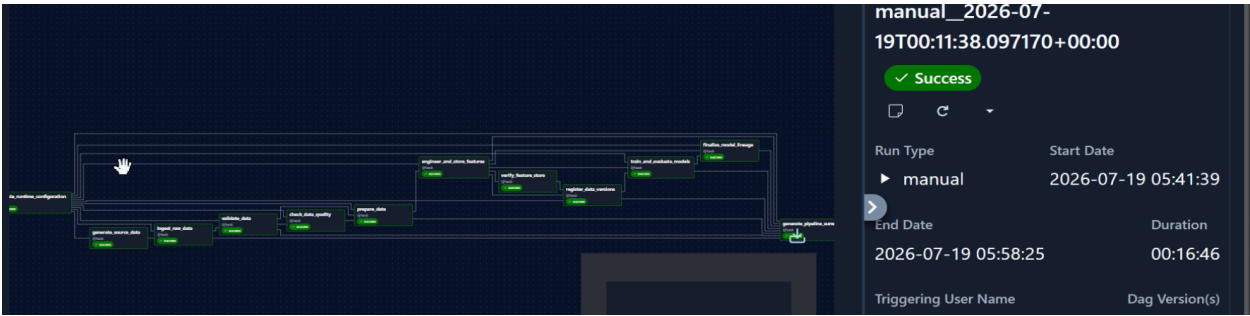
Raw Storage Structure 03_raw_storage_structure.png

	Run Name	Status	Type	Created at	Dataset	catalog_cov...	coverage	diversity	evaluated_
☐	 MODEL_2...		 Eval	1 day ago	-	0.072532699...	0.072532699...	0.941847782...	820
☐	 MODEL_2...		 Eval	1 day ago	-	0.637336504...	0.637336504...	0.982065088...	751
☐	 MODEL_2...		 Eval	1 day ago	-	0.057669441...	0.057669441...	0.959831092...	751
☐	 MODEL_2...		 Eval	1 day ago	-	0.637336504...	0.637336504...	0.982065088...	751
☐	 MODEL_2...		 Eval	1 day ago	-	0.057669441...	0.057669441...	0.959831092...	751
☐	 MODEL_2...		 Eval	1 day ago	-	0.637336504...	0.637336504...	0.982065088...	751
☐	 MODEL_2...		 Eval	1 day ago	-	0.057669441...	0.057669441...	0.959831092...	751
☐	 M	Go to the run	 Eval	1 day ago	-	0.637336504...	0.637336504...	0.982065088...	751
☐	 MO...		 Eval	1 day ago	-	0.057669441...	0.057669441...	0.959831092...	751

MLflow Results 11_MLRUNS.png



Repository Structure 91_repository_structure.png

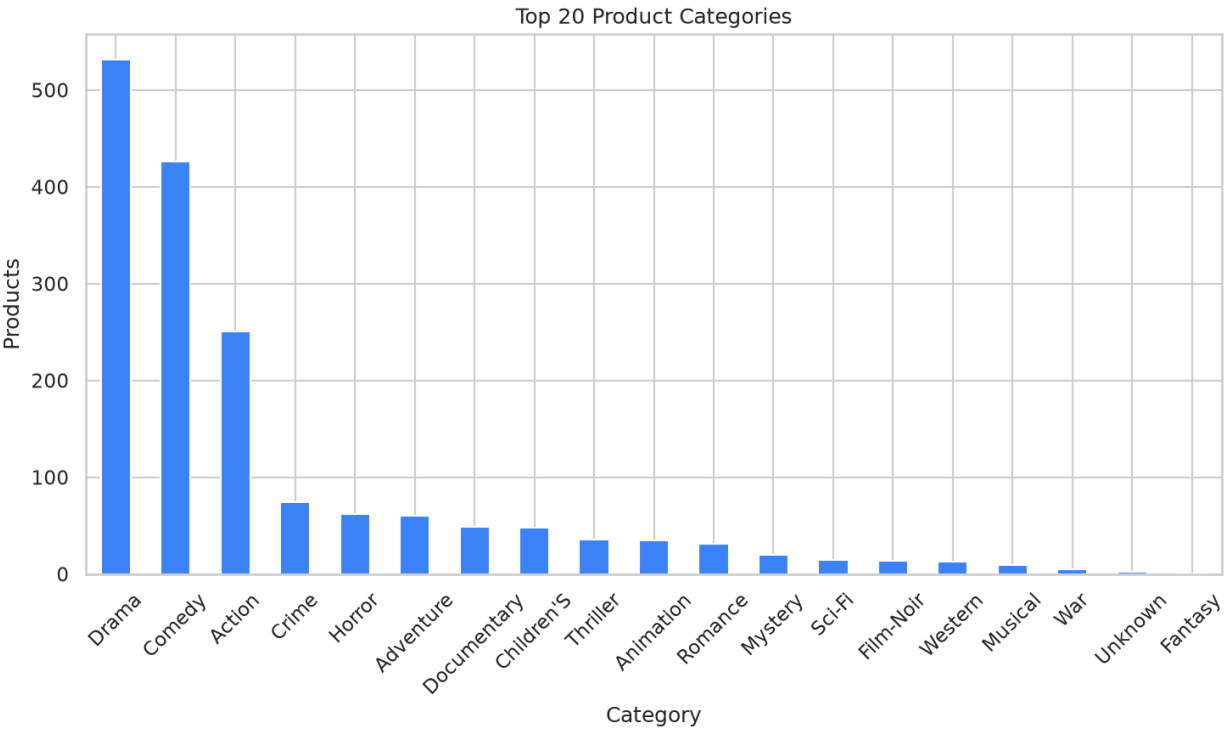


Airflow Orchestration 

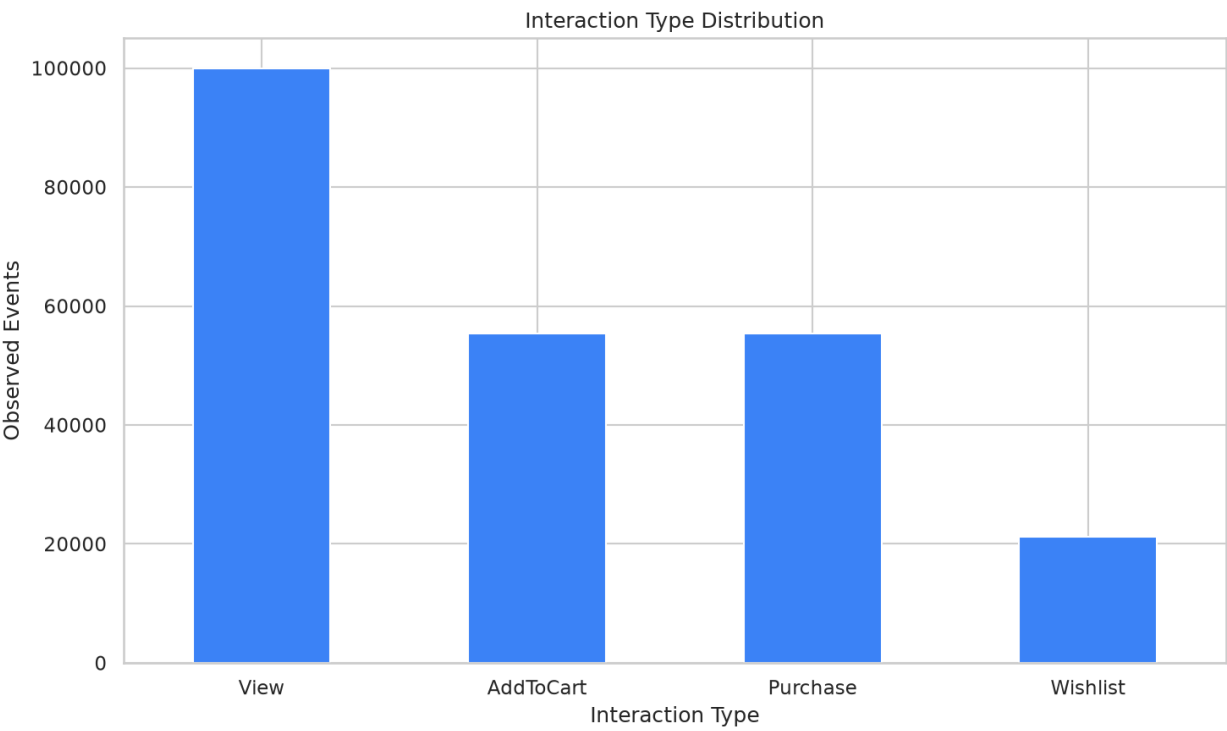
7.4 Latest Exploratory Data Analysis

Selected latest partitioned EDA batch: RECO_20260720_114419_07a4ea.

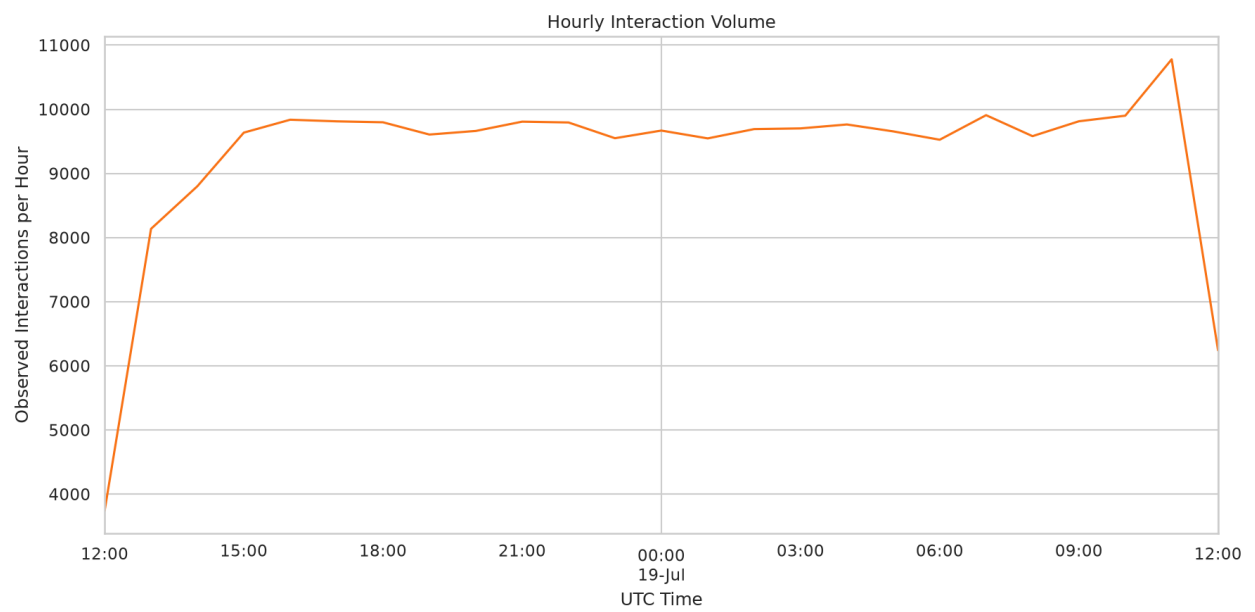
Measure	Value
Users	943
Products	1682
Interactions	231951
Density	0.06304669364224531
Sparsity	0.9369533063577546
Train / validation / test	162367 / 34792 / 34792



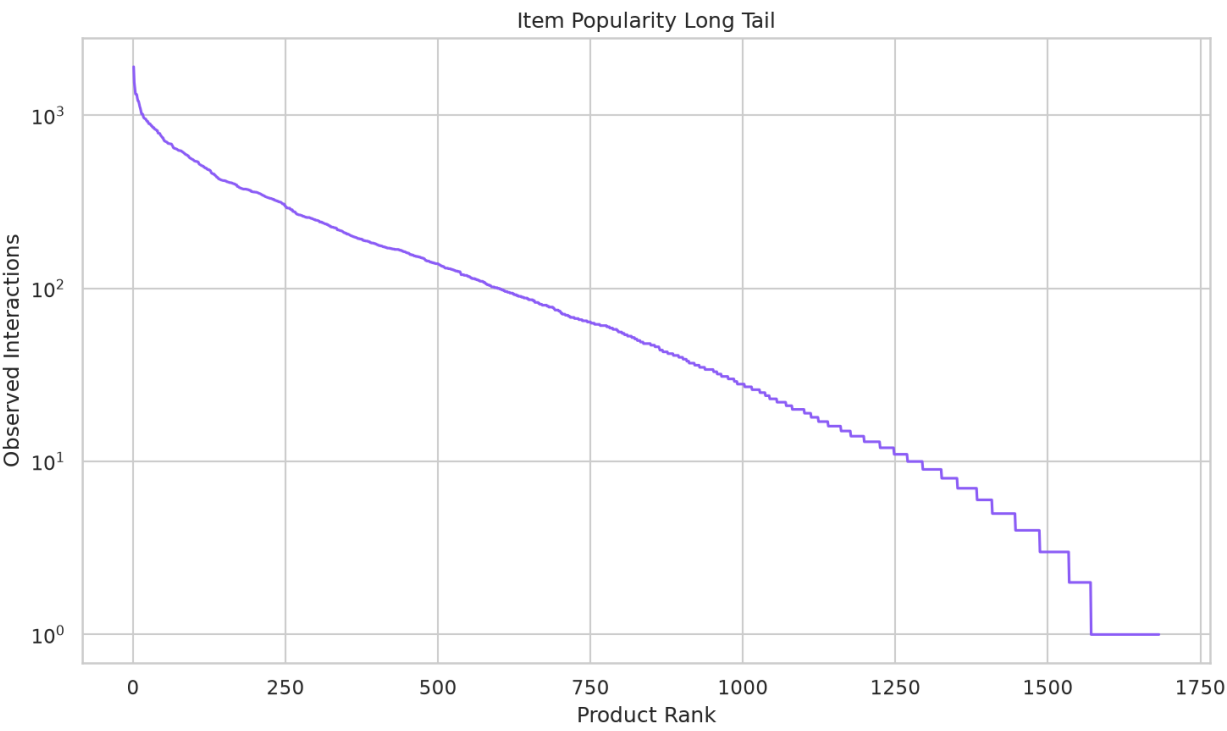
Category Distribution - EDA batch RECO_20260720_114419_07a4ea



Interaction Type Distribution EDA batch RECO_20260720_114419_07a4ea



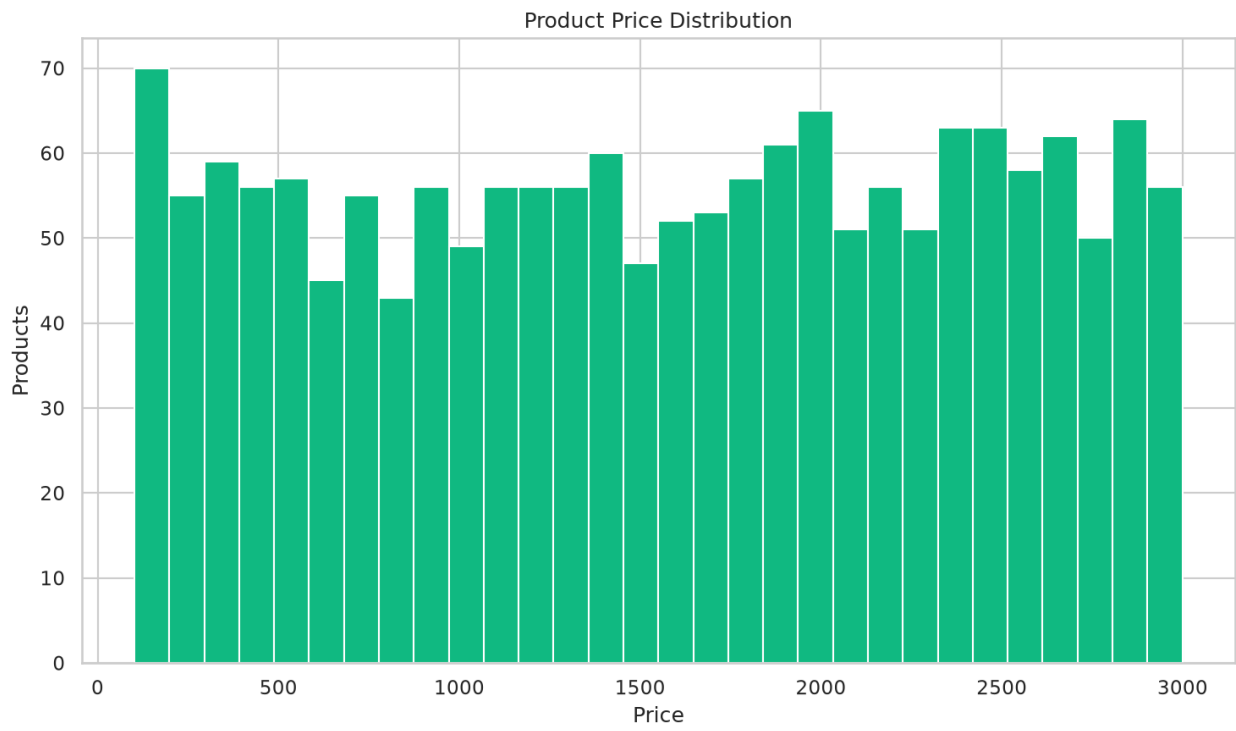
Interactions Over Time ÆfÂ¢Ã¢æšÂ¬Ã¢â¬Â¬ EDA batch RECO_20260720_114419_07a4ea



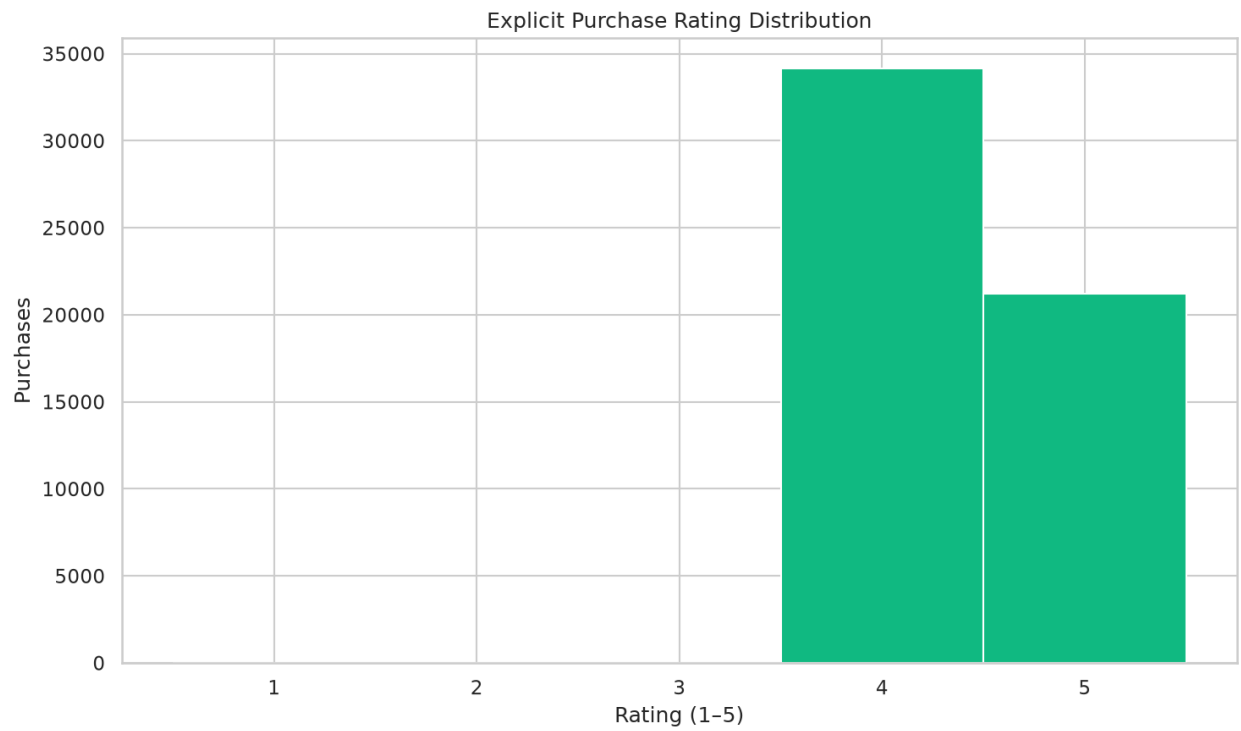
Item Popularity Distribution $\tilde{f}(\hat{c}_i)$ - EDA batch RECO_20260720_114419_07a4ea



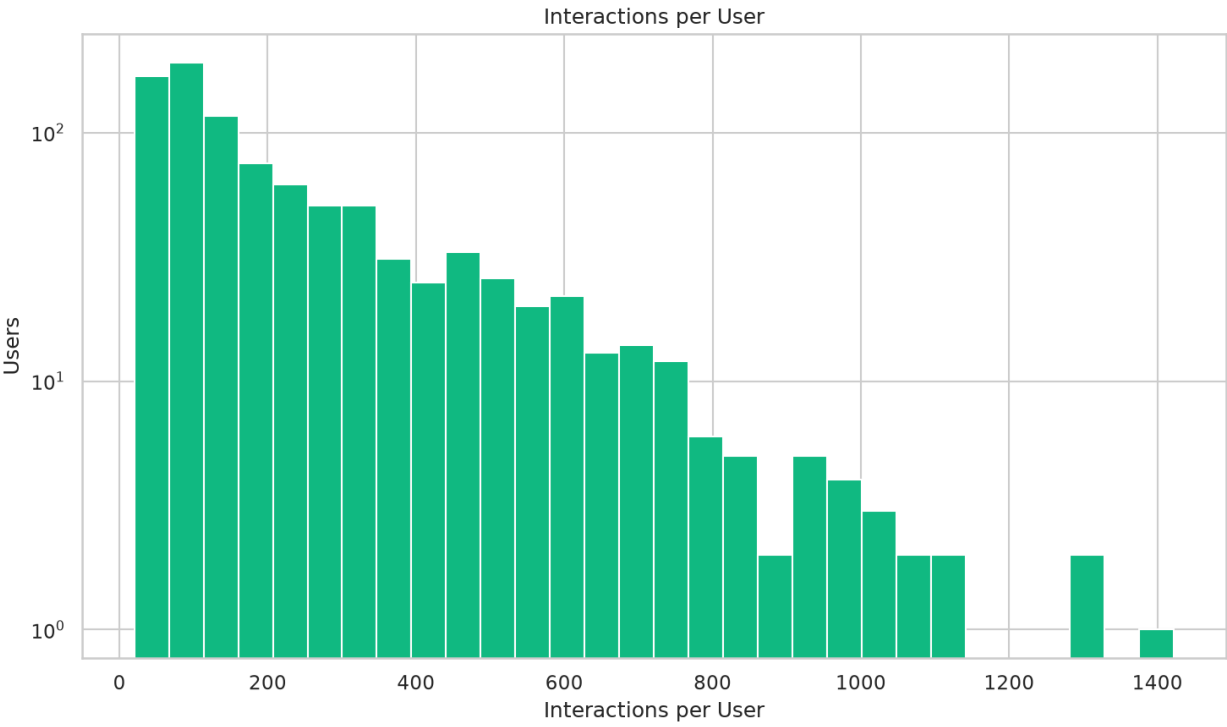
Numerical Correlation Heatmap ÆfÂcÂcâcŠ-Âcâ,-Â■ EDA batch RECO_20260720_114419_07a4ea



Price Distribution  EDA batch RECO_20260720_114419_07a4ea



Rating Distribution $\tilde{f}_{\mathcal{A}}(\hat{r})$ ■ EDA batch RECO_20260720_114419_07a4ea



User Interaction Distribution ÆfÂcÃcâ€šÂ~Ãcâ,-Â■ EDA batch RECO_20260720_114419_07a4ea



User Item Sparsity Heatmap ÆfÂ¢Ã¢â€šÂ¬Ã¢â,~Â■ EDA batch RECO_20260720_114419_07a4ea

8. Conclusion and Future Scope

8.1 Conclusion

A change is complete when its behavior is documented, typed, logged, tested, configuration-driven, and compatible with the layer contracts. Relevant unit tests and integration tests pass, failure modes are covered, and no secrets or machine-specific paths are introduced.

- Reliability: retries are safe, partial publications are not visible, and failed runs preserve diagnostic evidence. - Maintainability: packages have one responsibility and public interfaces are documented and typed. - Security: credentials never appear in code, YAML, logs, or artifacts. - Auditability: lineage and checksums establish what was processed and produced. - Portability: local MinIO/PostgreSQL/MLflow and cloud equivalents use the same contracts. - Scalability: interfaces permit replacing local Pandas processing when data volume outgrows a single machine.

End-to-End Automated Pipeline

Data Validation and Quarantine

Data Preparation and EDA

Recommendation Feature Engineering

Structured Feature Store

DVC Data Versioning and Lineage

Collaborative and Content-Based Models

MLflow Experiment Tracking

Apache Airflow Orchestration

Reproducible and Traceable Execution

8.2 Future Scope

The following items are future enhancements and are not claimed as completed functionality:

Kafka-based real-time ingestion

Cloud object storage

Online feature serving

Model registry and model promotion

Automated retraining

Data-drift monitoring

Model-drift monitoring

Recommendation API deployment

A/B testing

CI/CD integration

Kubernetes deployment

Production secrets management

Access control and monitoring